Universidad de La Sabana
Facultad de Ingeniería

Felix Mohr & Sergio Amortegui
2026.1

Fundamentos de Inteligencia Artificial

- Sheet 9-

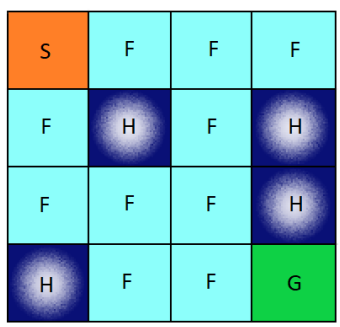
Environment & Rules. Use Python with only `numpy`. Do not use global random state; all randomness must come from a provided `numpy.random.Generator` named `rng`. Unless explicitly stated otherwise, functions that sample must accept a seed or `rng`. Keep the function signatures from the provided `.py` templates unchanged, as they will be tested automatically on Gradescope. You will only see public test outputs.

Files to implement. `solution.py` Do not rename files or functions.

Environment Considerations: Lake MDP (reuse). We consider the *Lake* environment on an $m \times n$ grid. Each cell is one of

`S` (unique start), `F` (free), `H` (hole, terminal), `G` (goal, terminal).

At each non-terminal step the agent chooses an action in $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$. With probability 0.8 the intended move is executed; with probability 0.1 each, the move slips to the two lateral directions if the state is not an absorbing one. Bumping against borders keeps the agent in place.
Rewards depend *only* on the state *entered*: a constant step reward in `F` (-0.1), a fixed reward in `H` (-1.0), and a fixed reward in `G` ($+1.0$). Upon entering any terminal cell, the process transitions to a synthetic absorbing state $s_\perp$ with only action $\perp$ and reward 0 thereafter.



We assume throughout that the discount factor satisfies $\gamma \in (0, 1]$.

Exercise 1 (Utility Methods – 2 Points) Implement the following helper functions to support the above classes:

1. `q_from_v(mdp, v, gamma)` Return $q^\pi(s, a)$ from $v^\pi(s)$ using:

$$q(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a)\, v(s').$$

2. `make_greedy_policy(mdp, v, gamma)` Return a dictionary mapping each state to its greedy action:

$$\pi(s) = \arg\max_a q(s, a).$$

3. `policy_mismatch(q_true, q_est)` Return a list of states where the greedy action under q^π differs from the one under q.

Exercise 2 (Value Iteration – 3 Points) Implement the class `ValueIteration`. The class should compute the optimal value function v^* and extract the corresponding greedy policy π^*. Implement the following:

1. **State Value Updates.** Perform value updates using the Bellman optimality equation:

$$v_{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} P(s'|s,a)\, v_t(s').$$

2. **Greedy Policy Extraction.** At convergence, return the greedy policy

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s'|s,a)\, v^*(s').$$

3. **Convergence Criterion.** Stop the iteration when

$$\|v_{t+1} - v_t\|_\infty < \varepsilon \cdot \frac{1-\gamma}{\gamma}.$$

Exercise 3 (Policy Evaluation Factory – 2 Points) Implement a `PolicyEvaluationFactory` class to support policy evaluation in both synchronous and asynchronous modes.

1. Implement `synchronous_update`, which updates all states in parallel using:

$$v(s) \leftarrow r(s) + \gamma \sum_{s'} P(s'|s,\pi(s))\, v(s').$$

2. Implement `asynchronous_update`, which only updates a given subset of states in each step.
3. Implement the method `step()`, which performs a single update step (sync or async).

Exercise 4 (General Policy Iteration – 3 Points) Implement the class `GeneralPolicyIteration` with configurable policy evaluation behavior.

1. The constructor should allow specifying:
   - Number of policy evaluation steps per improvement.
   - Whether updates are synchronous or asynchronous.
   - (Optional) Subsets of states to update per step.
2. The `run` method should:
   a) Perform partial policy evaluation using the factory.
   b) Improve the policy using a greedy policy over the current value estimates.
   c) Ensure that each evaluation-improvement cycle either strictly improves the policy or leaves it unchanged.
   d) Stop when the policy no longer changes.

Exercise 5 (Practice: Theoretical Exercises) These problems are not graded on Gradescope but are important for consolidating your theoretical understanding of the material. Write your answers by hand or in LaTeX.

1. **Convergence of Value Iteration.** Prove that Value Iteration converges to the unique fixed point v^* of the Bellman optimality operator. Show that this operator is a γ-contraction under the max norm, and explain why this guarantees convergence.

2. **Bellman Residual and Stopping Criterion.** Derive the bound for $\|v_t - v^*\|_\infty$ in terms of the Bellman residual $\|v_{t+1} - v_t\|_\infty$. Explain how this leads to a practical stopping condition in Value Iteration.